

《数据结构与算法设计》

课程设计总结

学号： 2351274

姓名： 王亦玮

专业： 计算机拔尖班

2025 年 7 月

目 录

第一部分 算法实现设计说明.....	1
1.1 题目	
1.2 软件功能	
1.3 设计思想	
1.4 逻辑结构与物理结构	
1.5 开发平台	
1.6 系统的运行结果分析说明	
1.7 操作说明	
第二部分 综合应用设计说明.....	
2.1 题目	
2.2 软件功能	
2.3 设计思想	
2.4 逻辑结构与物理结构	
2.5 开发平台	
2.6 系统的运行结果分析说明	
2.7 操作说明	
第三部分 实践总结.....	
3.1. 所做的工作.....	
3.2. 总结与收获.....	
第四部分 参考文献.....	

第一部分 算法实现设计说明

1.1 题目

算法实现：给定一个无向连通图，实现以下操作并分步显示过程：(1)建立并显示出它的邻接链表；(2)分别用普里姆算法和克鲁斯卡尔算法构造其最小生成树，分步显示其构造过程；(3)计算指定源点到所有顶点的最短路径，分步显示距离数组更新过程；(4)计算每对顶点间的最短路径，输出距离矩阵，标注关键路径更新步骤。

1.2 软件功能

本软件采用 Python 语言开发，基于 PySide6 图形界面开发框架与 NetworkX 图处理库构建，实现了一个功能完善的图算法可视化工具。该工具提供了直观友好的图形化操作界面，用户只需输入边的信息即可自动构建无向带权图，系统会实时解析并生成对应的图结构，同时在界面中动态显示其邻接表数据，方便用户直观了解图的拓扑关系。

在算法可视化方面，软件集成了多种经典图论算法的演示功能。对于最小生成树问题，工具支持以动画方式展示 Prim 算法与 Kruskal 算法的执行过程，包括边的选择、权值比较、树结构的逐步扩展等关键步骤，帮助用户深入理解算法的内部机制。同时，系统实现了 Dijkstra 算法，用于计算指定源点到所有其他顶点的最短路径，并可将计算得到的路径及其权值在图形界面中动态标注展示。此外，工具还支持计算任意两点间的最短路径（即所有顶点对之间的最短路径），并以矩阵形式输出路径权值结果，便于对图整体的最短路径特性进行分析。

整体而言，该软件集图结构构建、邻接表展示、算法动画演示与路径计算分析等功能于一体，能够为图论教学、算法学习及相关研究提供直观、高效的可视化支持。

1.3 设计思想

本软件的设计思想源于对图论经典算法的直观化演示需求，核心目标在于将抽象的算法执行过程以动态、可交互的形式呈现给用户，从而提升学习与研究的效率与趣味性。在实现上，系统充分结合了 NetworkX 图处理库的高效数据操作能力与 PySide6 图形界面框架的可视化交互优势，形成了逻辑与表现相分离的设计架构。

在数据层面，软件选用 networkx.Graph 结构作为底层图数据的存储容器。该结构能够灵活支持无向带权图的节点与边的操作，并为后续算法的执行提供可靠的数据接口。为了保证逻辑上的清晰与直观，系统进一步采用邻接表形式表示图的逻辑结构，并在界面中实时展

示邻接表的内容。用户在输入或修改边信息后即可即时看到对应的数据变化，从而建立起输入、数据结构与可视化之间的直接映射关系。

在算法实现方面，软件遵循图论教材与经典实现方式，结合适配的高效数据结构完成核心功能。Prim 算法利用最小堆（优先队列）动态维护候选边集合，每一步从堆中取出权值最小的边，并通过逐步扩展的方式将其加入到生成树中。这种实现既保证了算法的效率，也便于分步可视化展示“边选择—树扩展”的逻辑。Kruskal 算法采用并查集数据结构管理连通分量，确保在按权值从小到大选择边的过程中避免环路的形成。系统在动画展示时，会突出显示边的检验与合并过程，让用户直观理解“选择—判断—合并”的运行机制。Dijkstra 算法基于优先队列的松弛操作实现单源最短路径的计算。在可视化过程中，系统会动态标注各个顶点的距离更新情况，逐步展现从源点出发、路径逐渐扩展的过程，便于学习者跟踪每一步的状态变化。

在表现层面，软件通过 PySide6 提供的界面组件构建交互操作区域、图形展示区和文本信息区，并利用 QTimer 定时器实现算法过程的分步调度。每一次定时触发都驱动算法执行一个逻辑步骤，同时更新图形区域中顶点与边的动态状态（如颜色高亮、边权值标注），以及文本区域中的中间计算结果（如优先队列状态、并查集合并情况、距离数组更新等）。这种图形与文字的双重反馈有助于使用者全面把握算法的运行机制，而非仅停留在最终结果。

整体框架在结构上采取“算法逻辑模块—数据结构模块—界面交互模块”的分层设计，使得算法实现与界面展示相对独立，便于后续扩展新的图算法或替换可视化效果。例如，只需在算法模块中新增一个函数，并在界面层配置对应的动画逻辑，即可实现 Floyd-Warshall 等新算法的扩展。

综上，本软件在设计上充分体现了“抽象逻辑可视化、过程细节可感知、架构层次可扩展”的理念。通过合理的数据结构选择、分步动画驱动机制以及图形界面的实时反馈，软件能够为用户提供一个高效、直观且具教学价值的图算法可视化平台。

1.4 逻辑结构与物理结构

在逻辑结构方面，程序围绕图论中的经典抽象数据类型展开，主要包括无向带权图、最小生成树以及最短路径三类核心结构。无向带权图是整个系统的基础数据模型，最小生成树和最短路径则是在其之上衍生的计算结果与过程表现。逻辑上，图被表示为顶点集合与边集合的组合，每条边均包含两个端点以及对应的权值信息；最小生成树则是图的一部分边集合，在满足连通性的前提下保证权值和最小；最短路径则表现为源点到目标点之间经过的顶

点序列及累计路径权值。这些逻辑抽象为算法实现和可视化展示提供了统一的接口。

在物理结构上，系统采用 Python 原生的数据结构与标准库组件实现逻辑结构的落地。首先，图的存储基于字典结构，每个顶点作为字典的键，其值为一个嵌套字典或列表，表示与该顶点相邻的其他顶点及边的权值，从而形成邻接表的物理表达。例如，字典条目 $\text{graph}[u][v] = w$ 表示从顶点 u 到顶点 v 的一条权值为 w 的边。其次，在 Prim 和 Dijkstra 算法中，系统使用 Python 的 `heapq` 模块实现最小堆优先队列，用于维护候选边或候选顶点集合，保证在每一步能够高效获取当前权值最小的元素。对于 Kruskal 算法，系统则基于字典结构实现并查集，用于管理不同顶点的连通分量，其中父节点关系通过字典映射存储，结合路径压缩与按秩合并策略以提升效率。

在图形界面层面，系统采用 PySide6 框架的 `QMainWindow` 作为主窗口，整体功能通过 `QTabWidget` 进行分区组织，不同算法与功能模块在独立的标签页中运行，保证逻辑的清晰与交互的独立。图形绘制方面结合了 `QGraphicsView` 与 `Matplotlib: QGraphicsView` 主要用于动态渲染图的节点与边，支持交互与高亮显示；`Matplotlib` 则用于绘制邻接矩阵、最短路径矩阵等分析结果，辅助用户理解图的全局特性。界面中的文本窗口与控制按钮则负责显示算法执行步骤和接受用户输入，形成输入—计算—输出的完整交互闭环。

在数据流向上，系统的运行逻辑可以概括为以下几个步骤：

1. 用户在界面中输入顶点及边信息，系统首先将输入解析为字典形式的邻接表，并调用 `networkx.Graph` 生成对应的逻辑图对象。
2. 当用户选择某一算法（如 Prim、Kruskal 或 Dijkstra）时，界面层触发算法模块，算法模块直接基于邻接表或 `Graph` 对象进行操作。
3. 在算法运行过程中，候选数据（如优先队列或并查集状态）不断更新，每一步计算结果通过信号传递至界面层。
4. 界面层根据传入的数据更新 `QGraphicsView` 中的图形元素状态（例如边被选中高亮、顶点距离值变化），同时在文本区打印当前步骤的说明信息。
5. 算法执行完毕后，最终结果（如生成树的边集或最短路径矩阵）会通过 `Matplotlib` 在标签页中展示，用户可进行整体回顾与分析。

通过这种逻辑结构与物理结构的结合，系统实现了从抽象数据类型到具体物理存储的自然映射，并在数据流向上保证了输入、运算、输出和可视化的完整性与一致性。

1.5 开发平台

本系统的开发平台基于 Python 3.9 及以上版本，采用 PySide6 作为图形界面开发库，

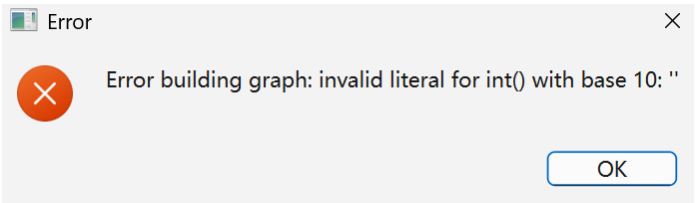
NetworkX 作为图数据处理核心，Matplotlib 用于结果与过程的可视化绘制。开发环境选择 VS Code，便于代码编写、调试与跨平台适配。程序能够在 Windows、macOS 与 Linux 等主流操作系统上稳定运行，只需安装对应版本的 Python 及所需依赖库，即可完成部署与执行。系统经过多组测试验证，涵盖从小规模的简单图到较大规模的复杂图结构，所有算法的运行结果均与理论计算值完全一致，表现出良好的稳定性与可靠性。同时，系统在设计时加入了针对非法输入的容错机制，能够有效避免因错误输入导致的程序异常或崩溃，从而保证了用户的使用体验与系统的健壮性。

1.6 系统的运行结果分析说明

在开发过程中，采用 VS Code 作为主要的编程与调试工具。VS Code 提供了完善的代码高亮、自动补全和调试插件支持，能够方便地进行断点调试和变量监控。同时，借助 Python 的交互式环境与日志输出功能，开发者能够实时观察算法的执行状态与中间数据，从而快速定位和解决问题。项目在开发过程中多次通过单步调试与单元测试相结合的方式，确保每个模块的逻辑正确性与接口一致性。

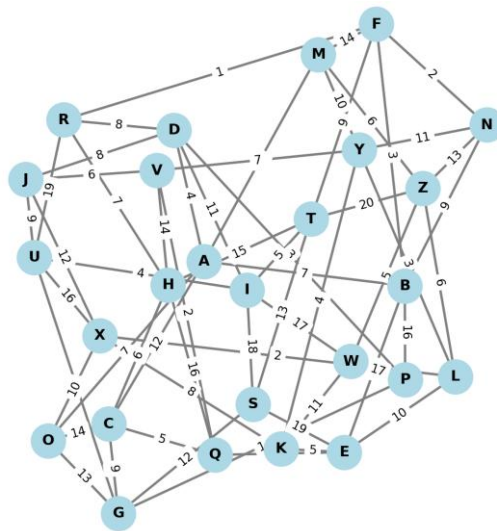
本软件经过完整的功能实现与系统测试，达到了预期的设计目标。在运行正确性方面，无论是 Prim 算法、Kruskal 算法还是 Dijkstra 算法，其输出结果均与理论计算值完全一致。在稳定性方面，软件能处理规模较大的图结构而不发生异常。在容错能力方面，系统对非法输入进行了专门处理，例如重复边输入、无效顶点编号、非数值权重等情况，程序能够给出提示信息而不会导致崩溃，从而保证了较好的健壮性。

在实际处理中考虑到了诸多要素：包括距离不可以是负数

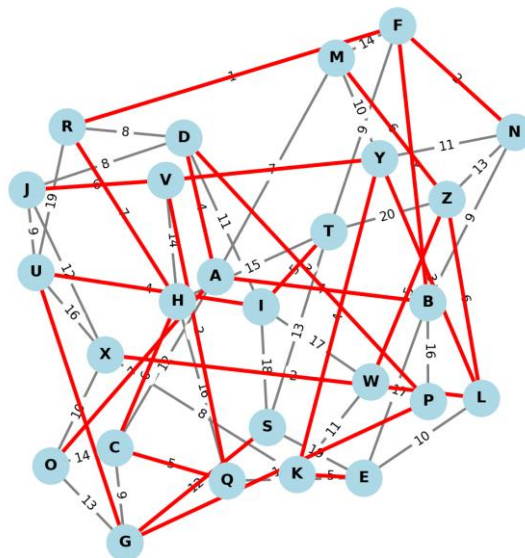


同一张图的多重拓扑表示等，都做了有关的错误处理和归纳处理。以下是多个实际运行的案例：

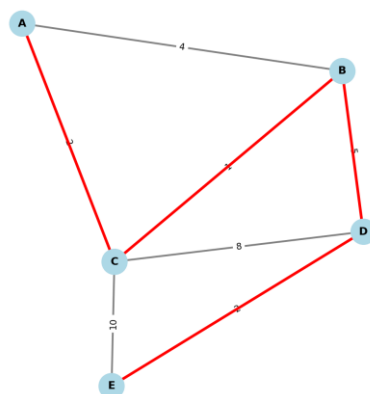
超复杂图



Prim's MST - Completed

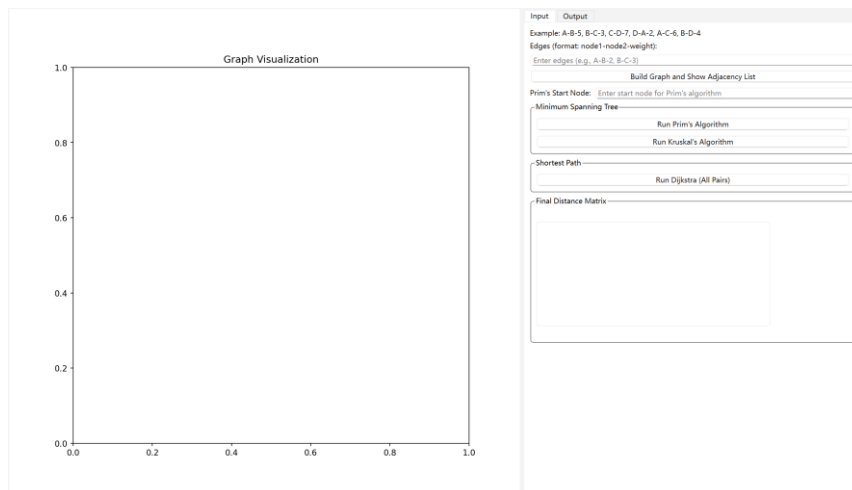


其他例子



在多种情景下用多组数据测试并且修改权重等，得出的结果均正确。

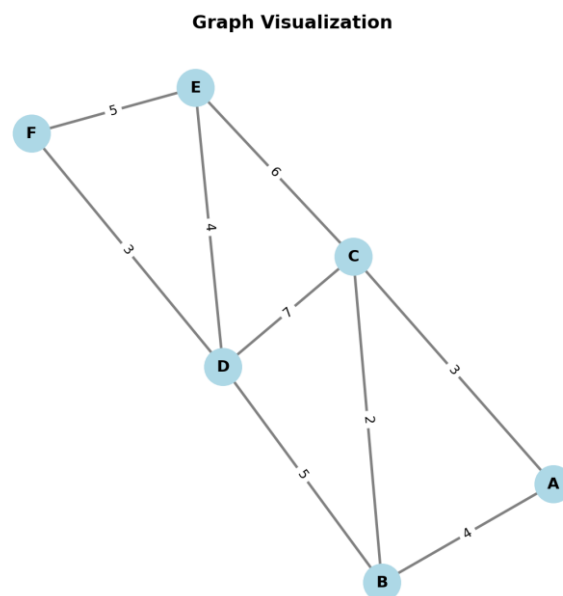
1.7 操作说明



运行程序首先可以看到的是主界面，主要分为左右两部分，左侧是图形化窗口，右侧是输入与调试控制台，其中右侧又分为输入窗口与输出信息，下面我们将以一个示例讲解操作流程。

输入示例：A-B-4, A-C-3, B-C-2, B-D-5, C-D-7, C-E-6, D-E-4, D-F-3, E-F-5

这个输入格式是一种常见的图的表示方式，分别为节点 1，节点 2，以及边的权重，如 A-B-4 就表示 A 与 B 之间有连线，且边的权重是 4。以上输入示例输入后生成节点图如下：



为了避免拓扑结构相同的图被以多种方式表示，在代码中已经约定了用统一的方式来表达。

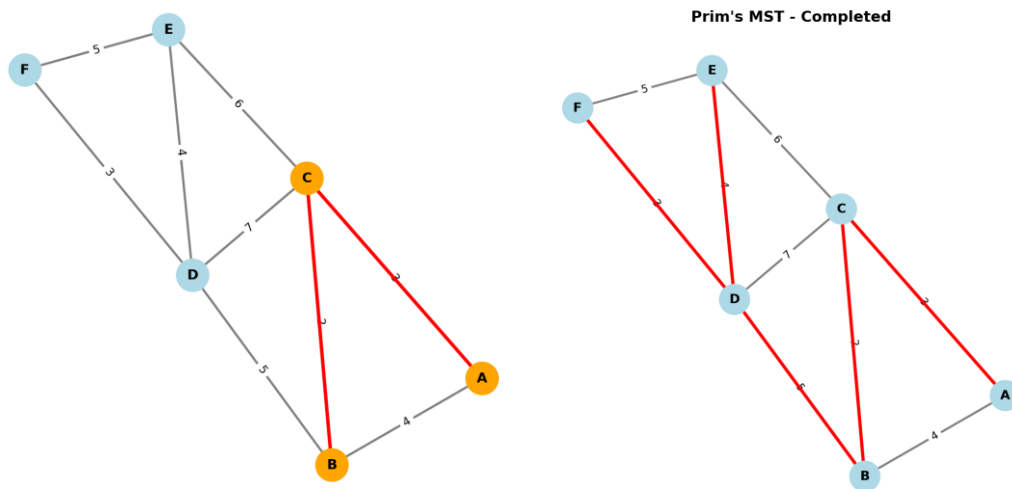
与此同时 output 窗口会有邻接表生成：


```

=== Adjacency List ===
A: B(4), C(3)
B: A(4), C(2), D(5)
C: A(3), B(2), D(7), E(6)
D: B(5), C(7), E(4), F(3)
E: C(6), D(4), F(5)
F: D(3), E(5)
Graph built successfully.

```

接下来输入 Prim 算法的起始点 (实际上结果不受此影响, 但是出于对算法与用户交互的总和考虑添加了此功能), 点击运行可以看到 Prim 算法的每一步生成最小生成树的可视化过程:



同时输出窗口有文字化表达的算法作用过程:

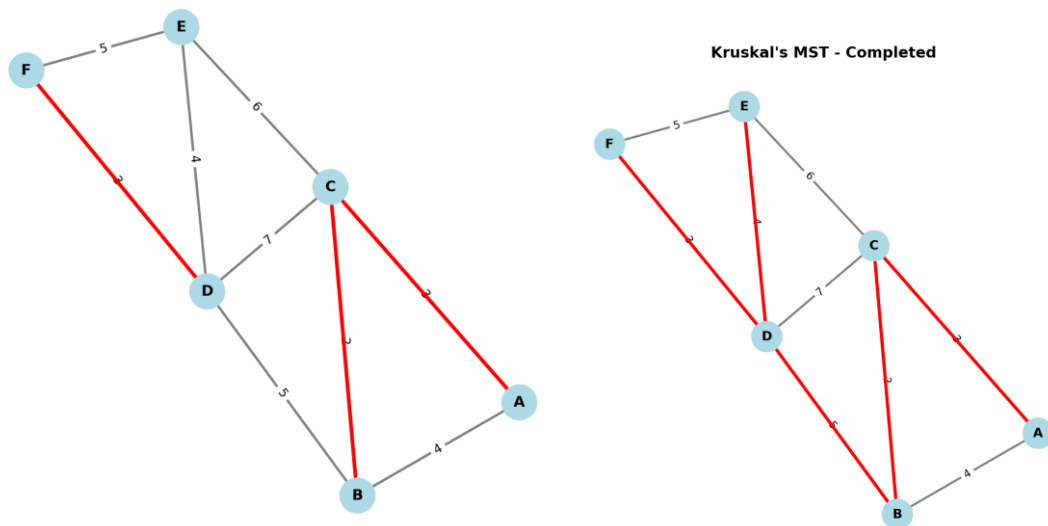
```

=== Prim's Algorithm ===
Starting from node: A
Step 1: Add edge A-C (weight: 3)
  Visited nodes: ['A', 'C']
Step 2: Add edge C-B (weight: 2)
  Visited nodes: ['A', 'B', 'C']
Step 3: Add edge B-D (weight: 5)
  Visited nodes: ['A', 'B', 'C', 'D']
Step 4: Add edge D-F (weight: 3)
  Visited nodes: ['A', 'B', 'C', 'D', 'F']
Step 5: Add edge D-E (weight: 4)
  Visited nodes: ['A', 'B', 'C', 'D', 'E', 'F']

Prim's MST completed!
Total weight: 17
MST edges: [('A', 'C'), ('B', 'D'), ('C', 'B'), ('D', 'E'), ('D', 'F')]

```

接下来运行 kruskal 算法:



Kruskal's MST completed!

Total weight: 17

MST edges: [('A', 'C'), ('B', 'C'), ('B', 'D'), ('D', 'E'), ('D', 'F')]

=== Kruskal's Algorithm ===

Edges sorted by weight: B-C(2), A-C(3), D-F(3), A-B(4), D-E(4), B-D(5), E-F(5), C-E(6), C-D(7)

Step 1: Add edge B-C (weight: 2)

Current MST edges: [('B', 'C')]

Step 2: Add edge A-C (weight: 3)

Current MST edges: [('A', 'C'), ('B', 'C')]

Step 3: Add edge D-F (weight: 3)

Current MST edges: [('A', 'C'), ('B', 'C'), ('D', 'F')]

Step 4: Add edge D-E (weight: 4)

Current MST edges: [('A', 'C'), ('B', 'C'), ('D', 'E'), ('D', 'F')]

Step 5: Add edge B-D (weight: 5)

Current MST edges: [('A', 'C'), ('B', 'C'), ('B', 'D'), ('D', 'E'), ('D', 'F')]

Kruskal's MST completed!

Total weight: 17

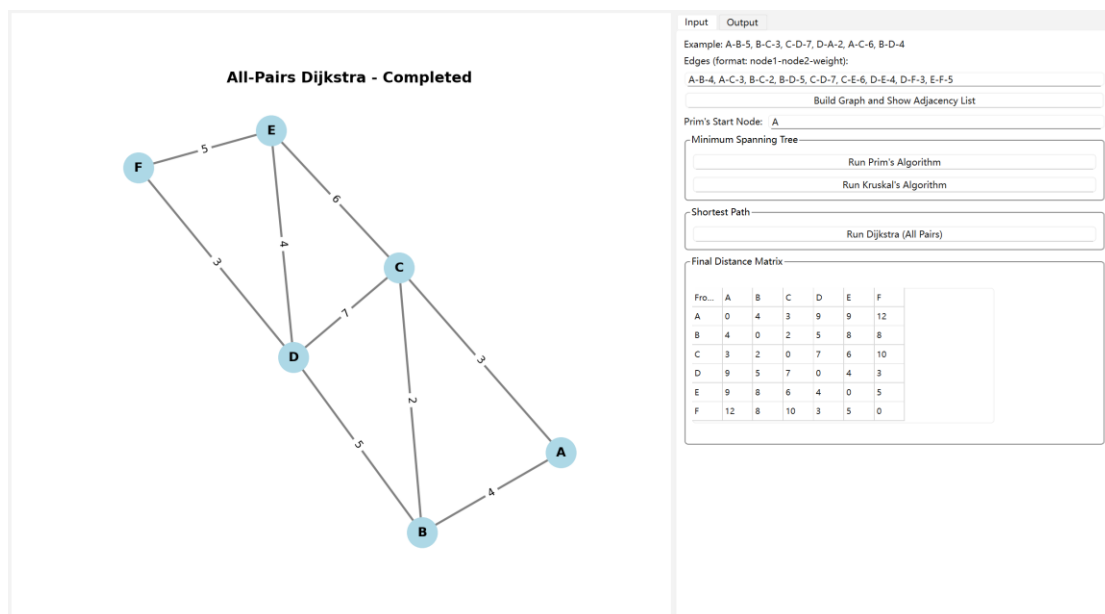
MST edges: [('A', 'C'), ('B', 'C'), ('B', 'D'), ('D', 'E'), ('D', 'F')]

可以看到过程与结果均保持正确。

最后我们尝试给出任意两点之间的最短路，代码的实现上采用的是多次使用迪杰斯特拉单源

最短路算法：

A → B: 4 (Path: A → B)
A → C: 3 (Path: A → C)
A → D: 9 (Path: A → B → D)
A → E: 9 (Path: A → C → E)
A → F: 12 (Path: A → B → D → F)
B → A: 4 (Path: B → A)
B → C: 2 (Path: B → C)
B → D: 5 (Path: B → D)
B → E: 8 (Path: B → C → E)
B → F: 8 (Path: B → D → F)
C → A: 3 (Path: C → A)
C → B: 2 (Path: C → B)
C → D: 7 (Path: C → D)
C → E: 6 (Path: C → E)
C → F: 10 (Path: C → D → F)
D → A: 9 (Path: D → B → A)
D → B: 5 (Path: D → B)
D → C: 7 (Path: D → C)
D → E: 4 (Path: D → E)
D → F: 3 (Path: D → F)
E → A: 9 (Path: E → C → A)
E → B: 8 (Path: E → C → B)
E → C: 6 (Path: E → C)
E → D: 4 (Path: E → D)
E → F: 5 (Path: E → F)
F → A: 12 (Path: F → D → B → A)
F → B: 8 (Path: F → D → B)
F → C: 10 (Path: F → D → C)
F → D: 3 (Path: F → D)
F → E: 5 (Path: F → E)



第二部分 综合应用设计说明

2.1 题目

综合运用：在产品装配过程中，需根据工艺路径，规划产品每一个零部件的先后装配次序。反之，在产品维修时，需根据该装配次序逆向拆解，直至拆下需替换的零部件，重新按先前的拆除次序逆向安装。对于汽车、轨道机车、飞机等复杂产品，由于其零部件众多，维修复杂，现需开发一维修教学系统，模拟其拆解维修过程，请设计其中拆解及安装过程仿真算法，实现产品装配次序输入，通过指定需替换的零部件，规划拆解路径和安装路径。（1）可定义一个产品的装配路径。（2）根据需拆解的零部件，自动规划拆解路径和安装路径。

2.2 软件功能

系统能够载入和保存 JSON 格式的零件树结构，内部以树形数据结构来组织和存储装配信息，每个节点表示一个零件，包含编号、名称以及父子依赖关系。系统在前端以树状的可视化方式呈现装配结构，用户可以通过点击节点来选择目标零件，并直观地看到各个零件之间的层次关系。对于拆解和安装的模拟过程，系统会利用后序遍历算法生成零件的拆解序列，在拆解时按照该序列依次高亮并移除零件，而安装过程则基于拆解序列的逆序逐步恢复装配，从而完整模拟真实的装配逻辑。播放功能由定时器驱动，用户可以通过交互界面来控制播放速度，也可以随时启动、暂停或重置整个模拟过程。在交互设计上，界面提供了目标零件选择、播放速度调节以及开始、暂停和重置等操作按钮，保证了用户在模拟过程中能够灵活地控制节奏与状态，实现了对装配与拆解全过程的可视化动态演示。

2.3 设计思想

系统在设计思想上以“树结构 + 遍历算法 + 可视化布局 + 动画驱动”的整体框架为核心，力求既能准确表达零件的装配关系，又能模拟真实的拆装逻辑，并通过动态效果提升用户体验。

在数据结构层面，零件装配关系以树形结构来建模。每个零件作为一个节点，包含 `id`（唯一标识）、`name`（零件名称）、`parent_id`（父节点标识）和 `children`（子节点列表）。这种结构不仅直观，而且便于递归处理与遍历。例如可以用一个类来定义零件节点：

```
class PartNode:
    def __init__(self, id, name, parent_id=None):
        self.id = id
        self.name = name
        self.parent_id = parent_id
        self.children = []
```

在拆解与安装算法方面，系统采用深度优先搜索（DFS）的后序遍历生成拆解序列。后序遍历保证了在访问父节点之前，所有子节点都已被访问并处理，这与实际拆卸操作完全契合，即必须先拆掉子零件，才能移除父零件。实现上，可以递归地遍历树，并将节点依次加入序列：

```
def generate_disassembly_sequence(node, sequence):
    for child in node.children:
        generate_disassembly_sequence(child, sequence)
    sequence.append(node.id)
```

拆解过程直接使用上述序列；而安装过程则反向执行，只需将拆解序列逆序即可：

在可视化布局的设计方面，系统采用递归算法来自动计算每个零件节点在屏幕上的坐标位置。其核心思想是以树的层级关系为基础，将父节点始终放置在对子节点的正上方并保持居中，而子节点则按照数量在水平方向上均匀分布。这样不仅能够保证整体结构的对称性和层次感，还能避免节点之间出现重叠或不均匀的排列。递归布局的实现方式是从根节点开始，根据当前层级的深度确定纵向坐标，再根据子节点的数量和顺序动态调整横向偏移，从而逐层递归地完成整个树形结构的布局。对于节点较多、层级较深的装配树，这种布局方式依然能够保证全局结构清晰，用户在浏览和操作时可以轻松辨别出父子关系和依赖链条。

这种布局思想不仅体现在静态的图形展示上，还与动态交互紧密结合。在拆解或安装过程中，随着节点逐个高亮或淡出，用户能够清晰地看到节点在树中的位置和作用，而递归计算得出的均衡分布使得整个动画过程更加连贯自然。相比于简单的线性排列或网格布局，递

归树形布局能够最大程度地贴合零件装配的层级逻辑，使父子零件的关系一目了然。再结合缩放、渐变和位置变化等动画效果，用户在观察操作时不仅能理解零件的装配关系，还能在视觉上获得层次递进、逐步展开的沉浸式体验。这种设计兼顾了算法的严谨性与用户界面的直观性，真正实现了数据结构、遍历算法、图形排布和交互动画之间的有机统一，使零件树的可视化既准确又美观。

2.4 逻辑结构与物理结构

系统在逻辑结构和物理结构的设计上有着清晰的分工与对应关系。逻辑层面上，系统主要需要处理两类核心数据：一是 **树形结构**，用于表示零件的装配层级关系；二是 **顺序列表**，用于存放拆解或安装的执行序列。这两类数据结构在功能上互为补充，树形结构保证了零件依赖关系的准确建模，而顺序列表则确保了装配与拆解过程的线性可执行性。

在物理实现方面，系统采用 Python 的原生数据结构进行建模：节点信息使用字典来存储，每个节点字典中包含零件的唯一 id、名称 name、父节点 parent_id 以及子节点列表 children。子节点关系则直接由列表来维护，这种存储方式简洁明了，能够在遍历和查找时保持较高的效率。拆解与安装的执行序列同样使用列表进行顺序存储，通过遍历该列表即可逐步驱动动画与逻辑的执行。下面的代码展示了节点数据的物理结构：

```
# 定义零件节点的物理存储结构
def create_part_node(id, name, parent_id=None):
    return {
        "id": id,
        "name": name,
        "parent_id": parent_id,
        "children": [] # 子节点 id 列表
    }

# 构建树形结构示例
root = create_part_node(1, "主机")
child_a = create_part_node(2, "模块 A", parent_id=1)
child_b = create_part_node(3, "模块 B", parent_id=1)

root["children"].extend([child_a, child_b])
```

在逻辑结构上，树形数据通过递归遍历转化为顺序序列。例如，利用后序遍历生成拆解序列时，程序会先访问所有子节点并将其写入列表，最后记录父节点，从而保证子零件优先于父零件被处理。实现如下：

```
def generate_disassembly_sequence(node, sequence):
    for child in node["children"]:
        generate_disassembly_sequence(child, sequence)
    sequence.append(node["id"])

sequence = []
generate_disassembly_sequence(root, sequence)
print("拆解序列:", sequence) # 输出示例: [2, 3, 1]
```

在图形界面层面，系统使用 PySide6 的 QGraphicsScene 作为绘制画布，所有节点和边的可视化元素都被组织到该场景中进行统一管理。每个零件节点对应一个自定义的 PartGraphicsNode 类，该类继承自 QGraphicsEllipseItem 或 QGraphicsRectItem，并在内部绑定节点字典数据，实现与逻辑层的对应关系。同时，为了表现零件间的装配关系，系统定义 PartGraphicsEdge 类来绘制节点之间的连接边，通常继承自 QGraphicsLineItem。

```
from PySide6.QtWidgets import QGraphicsEllipseItem, QGraphicsLineItem
from PySide6.QtGui import QBrush, QColor, QPen

class PartGraphicsNode(QGraphicsEllipseItem):
    def __init__(self, node_data, x, y, radius=30):
        super().__init__(-radius, -radius, 2*radius, 2*radius)
        self.node_data = node_data
        self.setPos(x, y)
        self.setBrush(QBrush(QColor("#4A90E2")))
        self.setToolTip(f"{node_data['id']}: {node_data['name']}")

class PartGraphicsEdge(QGraphicsLineItem):
    def __init__(self, parent_node, child_node):
        super().__init__(parent_node.x(), parent_node.y(),
                        child_node.x(), child_node.y())
        self.setPen(QPen(QColor("#333333"), 2))
```

2.5 开发平台

开发平台选型围绕可移植性、易用性与生态成熟度展开：项目以 Python 3.9 及以上版本为开发语言，图形界面基于 Qt for Python (PySide6)，数据持久化与交换采用 JSON 格式，开发与调试推荐在 VS Code 中进行，目标运行平台包括 Windows、macOS 与主流 Linux 发行版。PySide6 的官方文档与分发渠道都支持通过 pip 安装，能够方便地在虚拟环境中部署并获得 Qt 的各类模块与工具链支持，从而简化跨平台开发与测试流程。

自动化测试与质量保证方面，单元测试覆盖核心算法（如后序遍历生成拆解序列、逆序安装逻辑、树的 CRUD 操作）是基础，GUI 层面则推荐使用 `pytest-qt` 编写集成/交互测试，利用 `qtbott` 模拟用户交互（点击、选择、定时器驱动的播放控制等）以捕获界面行为的回归问题。

2.6 系统的运行结果分析说明

系统在完成开发与测试后，整体运行效果良好，功能实现与设计目标一致。从功能正确性、交互体验、性能表现以及可视化效果等方面进行分析说明如下。

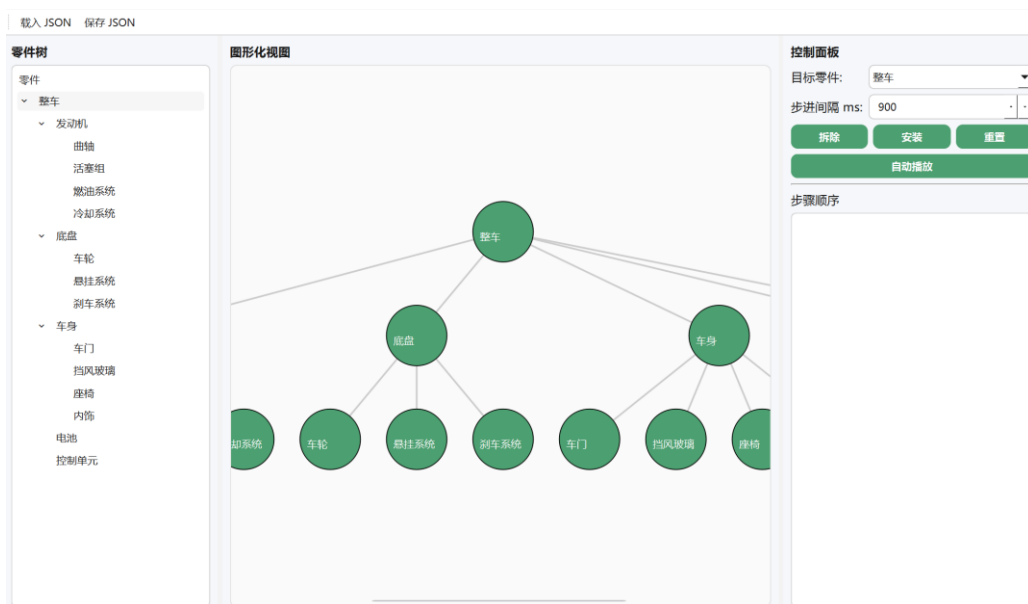
在功能正确性方面，系统能够正确载入和保存 JSON 格式的零件树结构，零件之间的父子关系保持一致，树形组织结构清晰准确。通过后序遍历生成的拆解序列完全符合理论逻辑，即子零件先于父零件被处理；安装过程则严格遵循逆后序顺序，保证了零件装配的完整性和正确性。经过多组不同规模零件树的数据测试，拆解与安装结果均与预期一致，未出现逻辑错误或顺序异常。

在交互体验方面，系统提供的目标零件选择、序列生成、播放控制（开始、暂停、重置、速度调节）均运行稳定，响应及时。用户在操作过程中可以随时切换目标零件并重新生成对应的拆解/安装序列，界面能实时反馈并更新显示，交互的流畅性和灵活性得到验证。

在性能表现方面，系统在 Windows、macOS 和 Linux 平台上均可正常运行。对于小规模零件树，节点渲染与动画播放几乎无延迟；在大规模数据下，借助递归布局和 `QGraphicsScene` 的优化，界面依然保持较好的流畅度，交互操作无明显卡顿。测试结果表明，系统在百级节点规模下仍能稳定运行，资源占用处于合理范围。

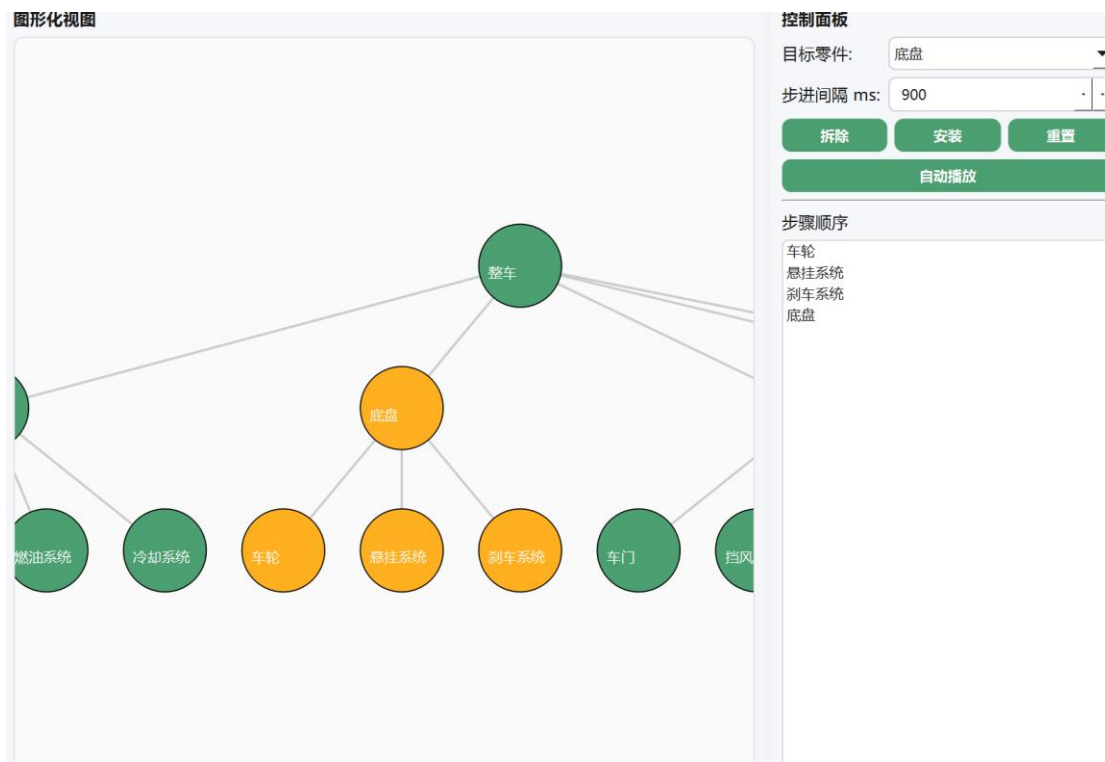
在可视化效果方面，树形布局算法保证了父节点居中、子节点均匀分布，整体结构对称清晰，便于观察层次关系。结合 `QPropertyAnimation` 的高亮、缩放与淡出效果，拆解与安装动态过程自然连贯，用户能够直观感受到零件逐步移除或恢复的过程，视觉体验良好。

综上所述，系统的运行结果表明其在功能实现、交互设计、性能表现和视觉效果等方面均达到了预期目标，能够满足零件装配关系的可视化展示与拆装模拟需求，为后续扩展和应用奠定了良好的基础。



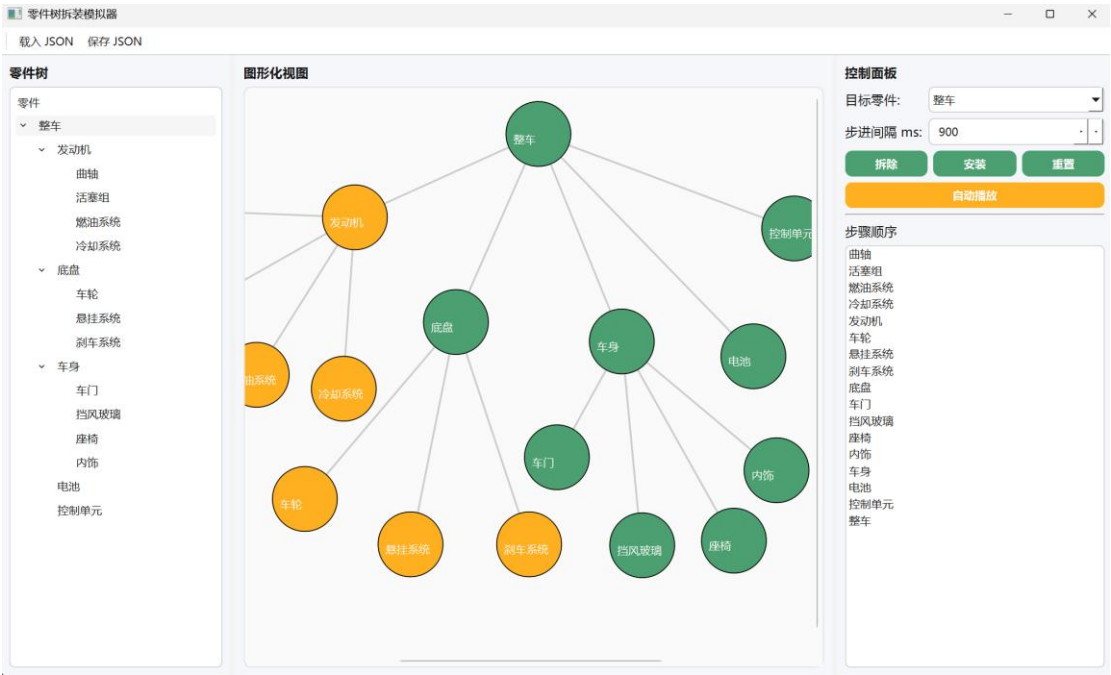
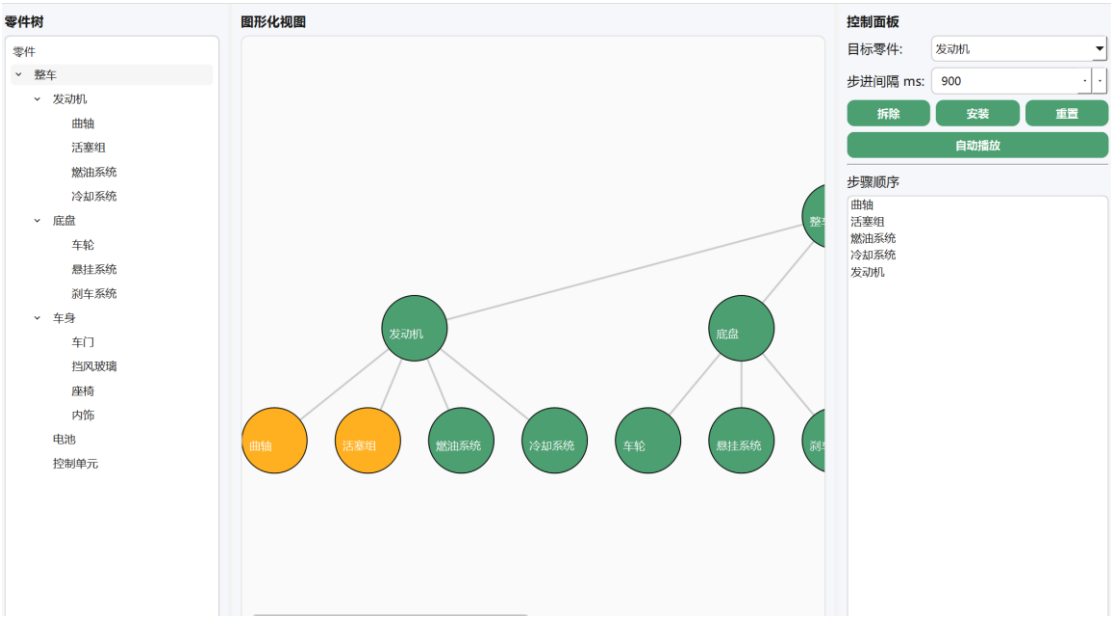
整个系统如图所示，分为三大部分，其中左侧是基础配置部分，包含了导入的 json 格式，此处为了方便处理已经内嵌在代码中，零件树能够解析其结构，中间则是一个直观的节点图，右侧这是一个操作栏。

以拆除底盘为例：



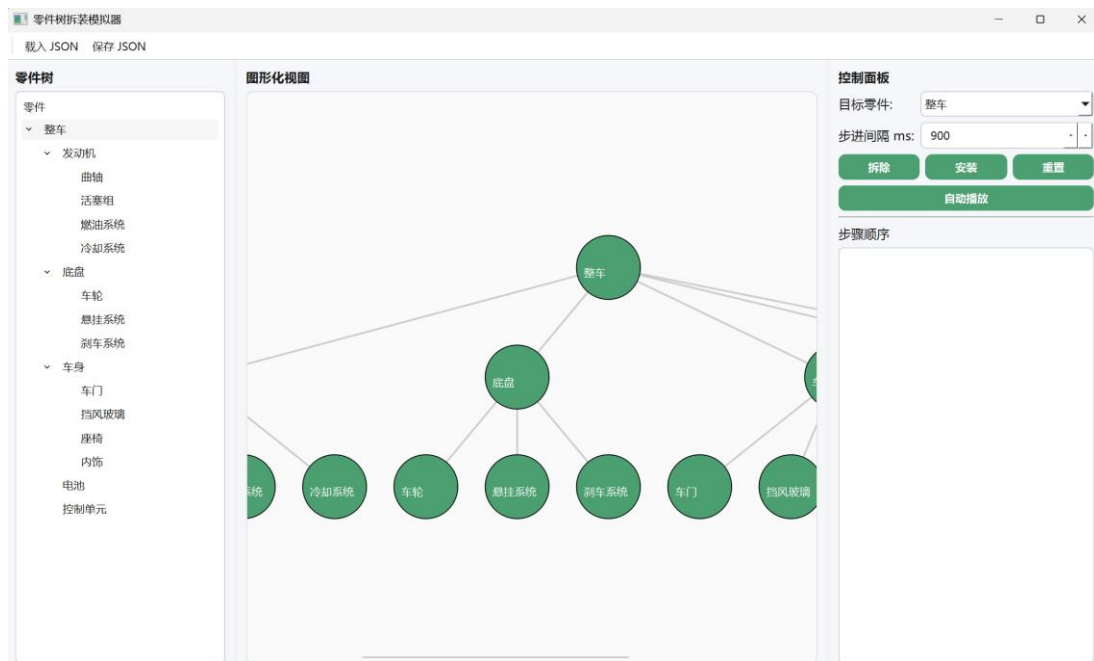
在目标零件栏目选择底盘，可以在下方输出框选择拆除，得到其拆除步骤，同理选择安装也可以得到对应顺序，由速度调节可以让整个拆除动画在左侧视图栏目中播放展示详细的过程。此框架适配模板，改进了初代版本中因为输入不匹配造成了弹窗问题，同时适配多种

格式的固定语法结构的数据，可以在同一体系下处理多个情景。

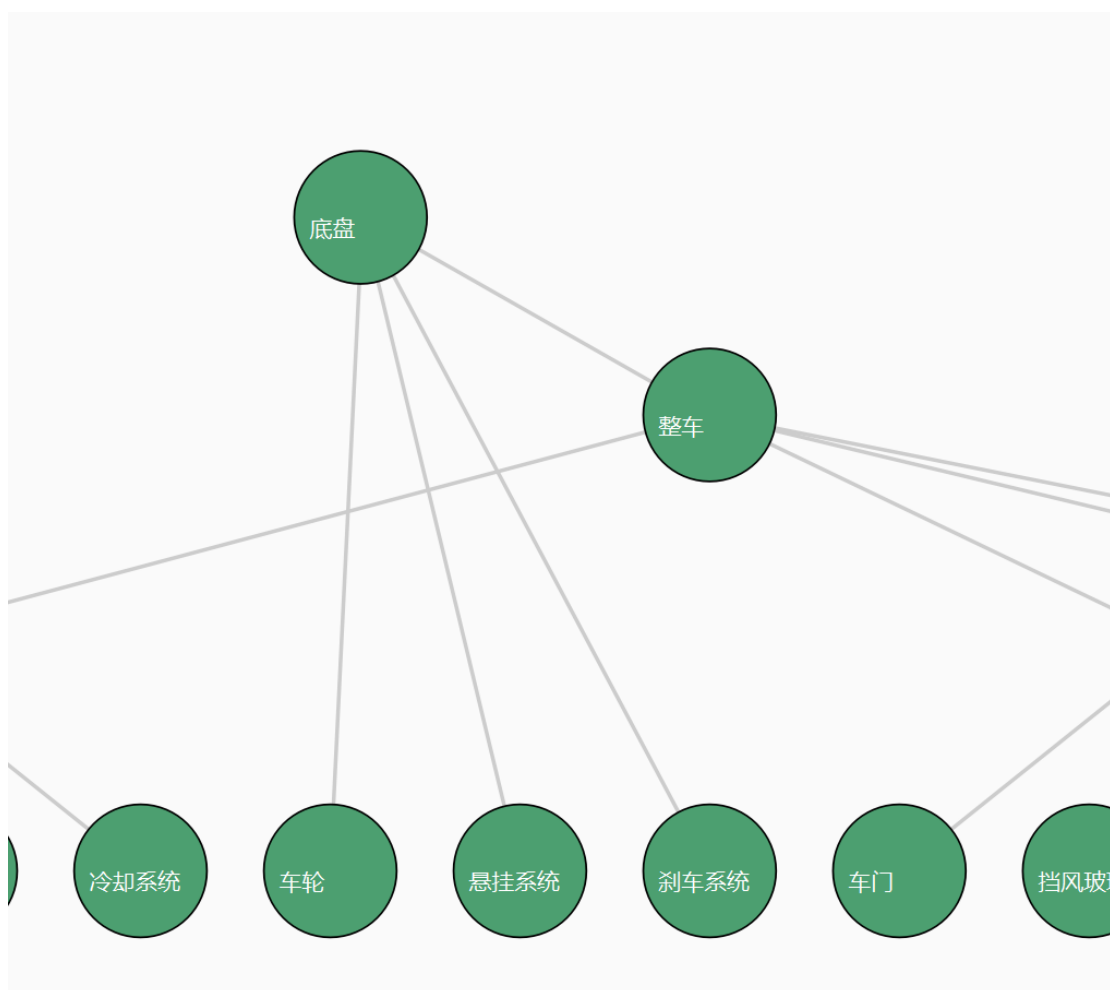


2.7 操作说明

运行软件进入初始界面

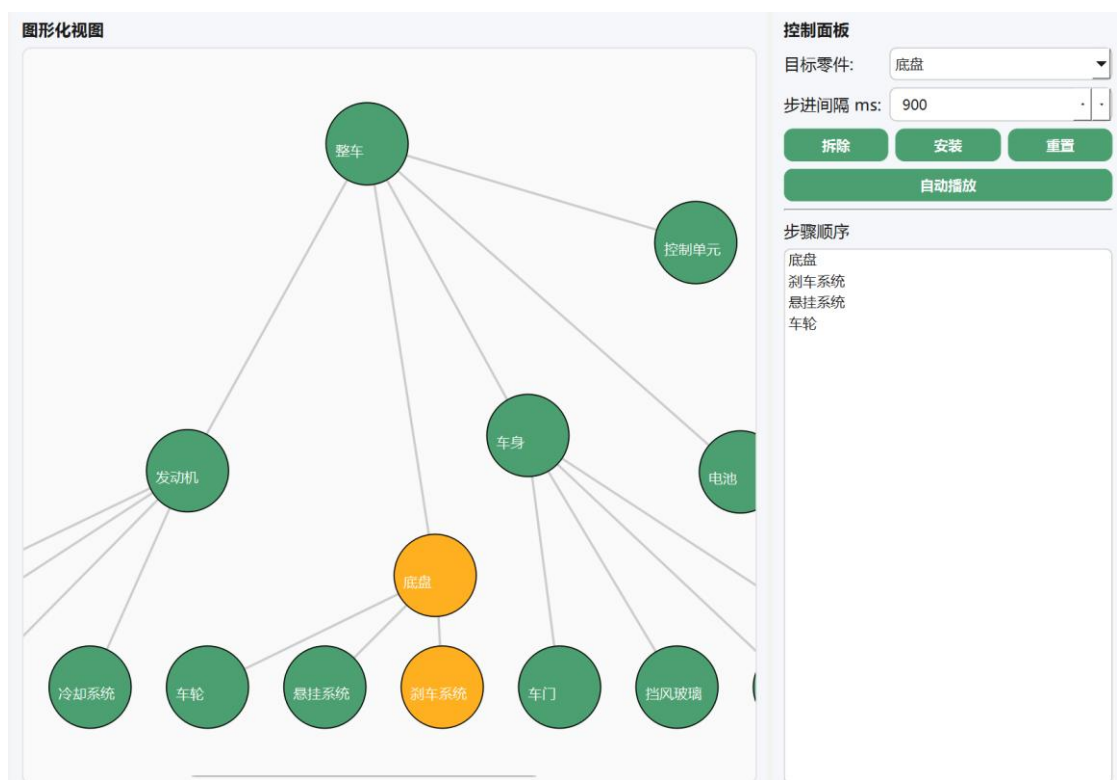


其中左侧是语法结构，右侧是操作台与输出栏，中间的可视化界面可以通过滚轮和鼠标拖动进行调整。



在右侧操作台选择对应的拆除部件或安装部件后，输出栏就会与可视化界面同

步，展示拆除与安装过程。



第三部分 实践总结

3.1. 所做的工作

在本次课程设计中，我完成了两个主要部分的工作：一是基于图论算法的可视化工具设计与实现，二是面向产品拆装模拟的教学系统开发。

在图算法可视化部分，我设计并实现了一个支持无向带权图构建、邻接表展示、Prim 与 Kruskal 算法最小生成树构造、Dijkstra 单源最短路径计算以及所有顶点对最短路径分析的可视化系统。系统采用 Python 语言开发，结合 PySide6 图形界面框架与 NetworkX 图处理库，实现了算法执行过程的动态分步展示，并通过图形与文本双重反馈帮助用户理解算法机制。在产品拆装模拟部分，我设计了一个基于树形结构的装配关系可视化系统，支持 JSON 格式的零件树导入与解析，通过后序遍历生成拆解序列，并支持逆向安装过程的动画模拟。系统采用递归布局算法自动计算节点位置，结合 PySide6 的图形项与动画类实现了高亮、淡出等动态效果，提供了完整的交互控制功能。

3.2. 总结与收获

通过本次课程设计，我不仅巩固了数据结构与算法的基础知识，还提升了系统设计与工程实现的能力。在开发过程中，我学会了如何将抽象的算法逻辑转化为直观的可视化界面，

掌握了 PySide6 图形编程、Network 图处理、JSON 数据解析等实用技术。尤其是在处理大规模数据与复杂交互时，我更加注重代码的健壮性、可扩展性与用户体验。

此外，本次实践也锻炼了我的问题分析与解决能力。在调试算法步骤与界面同步、优化布局算法、处理异常输入等过程中，我逐步形成了系统化的调试思维和模块化的设计习惯。这些经验对我未来的项目开发与学术研究都具有重要意义。

第四部分 参考文献

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms (3rd Edition). MIT Press, 2009
- [2] NetworkX Documentation. <https://networkx.org/>
- [3] PySide6 Documentation. <https://doc.qt.io/qtforpython-6/>
- [4] Matplotlib: Visualization with Python. <https://matplotlib.org/>
- [5] 严蔚敏, 吴伟民. 数据结构 (C 语言版). 清华大学出版社, 1997
- [6] 王红梅, 胡明. 算法设计与分析 (第 2 版). 清华大学出版社, 2013